UNIVERSIDAD DE COSTA RICA
Facultad de Ingeniería
Escuela de Ingeniería Eléctrica

Trabajo final de graduación
Licenciatura

Diseño e Implementación de un Acelerador de Hardware con FPGA para el Algoritmo de Redes Neuronales "Forward-Forward"

por

Yehohnathan Miranda Vigil

Ciudad Universitaria Rodrigo Facio

Julio del 2025

Acta de Defensa

Derechos de propiedad intelectual

Dedicatoria

Prefacio el autor

Reconocimientos

Tabla de contenidos

Fe de erratas . . . . . **IX**

Tabla de figuras . . . . . **X**

Tabla de cuadros . . . . . **XI**

Nomenclatura . . . . . **XII**

Resumen . . . . . **XIV**

Abstract . . . . . **XV**

1 Introducción **1**

1.1 Contexto 1

1.2 Antecedentes 4

1.3 Planteamiento del Problema 6

1.4 Justificación 8

1.5 Objetivos 9

1.6 Metodología 10

1.7 Organización del documento 11

2 Marco Teórico **12**

2.1 Fundamentos de las Redes Neuronales Artificiales 12

2.1.1 Del perceptrón simple al perceptrón multicapa 12

2.1.2 Diversificación de las ANN 16

2.2 Redes Neuronales Binarias 18

2.3 Redes Neuronales Convolucionales 21

2.4 Algoritmos de Entrenamiento: Retropropagación y Forward-Forward 25

2.4.1 Retropropagación 25

2.4.2 Forward-Forward: Una Alternativa al Entrenamiento Convencional . . 26
2.5 Aceleradores de Hardware: Rol de las FPGAs 30

3 Desarrollo del Proyecto **34**
3.1 Evaluación Integral del Acelerador de Hardware 34
3.1.1 Calidad en Clasificación de Redes Neuronales 34
3.1.2 Rendimiento: Throughput, Latencia y Recursos 35
3.1.3 Eficiencia Energética en la FPGA 36
3.2 Diseño de la Arquitectura con HLS 37

4 Conclusiones y recomendaciones **38**

Fe de erratas

Tabla de figuras

1 Estructura de una neurona [27]. 13

2 Estructura de un Perceptrón [29]. 14

3 Estructura de un Perceptrón Multicapa [32]. 15

4 Estructura de una BNN [40]. 21

5 Una CNN entrenada para extraer características y clasificar números escritos a mano. El proceso incluye la aplicación de convoluciones sobre la imagen original del número nuevo, seguido de un pooling para reducir el tamaño de la convolución resultante [35]. 22

6 Algoritmo de entrenamiento: Retropropagación [48]. 26

7 Algoritmo de entrenamiento: Forward-Forward [48]. 27

8 Artix-7 FPGA Nexys A7. 31

Tabla de cuadros

1 Tabla de verdad de la compuerta XNOR ($\oplus$) con valores 1 y -1, para una BNN. 18

Nomenclatura

Siglas

ANN Red Neuronal Artificial, modelo computacional inspirado en la estructura y funcionamiento del cerebro humano, compuesto por neuronas artificiales interconectadas que procesan información en capas para realizar tareas como clasificación y reconocimiento de patrones.

BNN Red Neuronal Binarizada, un tipo de red neuronal artificial en la cual los pesos y las activaciones se representan de forma binaria (por ejemplo, -1, +1 o 0, 1). Son especialmente adecuadas para aplicaciones en tiempo real y sistemas embebidos, donde la eficiencia energética es crucial.

CNN Red Neuronal Convolucional, un tipo de red neuronal artificial especializada en el procesamiento de datos con una estructura de cuadrícula, como imágenes, utilizando capas de convolución para extraer características y capas de pooling para reducir la dimensionalidad.

CPU Unidad central de procesamiento, componente principal del ordenador que ejecuta instrucciones y realiza cálculos.

FF Algoritmo Forward-Forward, un método de entrenamiento para redes neuronales que utiliza datos positivos reales y datos negativos sintéticos para optimizar las representaciones.

FPGA Matrices de compuertas programables en campo, circuito integrado que puede configurarse para desempeñar una función específica.

GPU Unidad de procesamiento gráfico, componente especializado en procesamiento gráfico y cálculos paralelos, ideal para tareas intensivas.

HDL Lenguaje de descripción de hardware, utilizado para describir la estructura y el comportamiento de circuitos electrónicos, como las FPGAs.

HLS Síntesis en alto nivel, herramienta presente en Vivado que permite convertir especificaciones escritas en lenguajes de alto nivel como C++ en implementaciones a nivel de transferencia de registros (RTL).

AI Inteligencia artificial, campo de estudio que se enfoca en crear sistemas capaces de realizar tareas que normalmente requieren inteligencia.

J Joules, unidad de medida de energía en el Sistema Internacional de Unidades (SI), equivalente a un watt (W) por segundo (s).

ML Aprendizaje automático, rama de la inteligencia artificial (IA) que permite a las computadoras aprender de los datos sin una programación explícita.

RTL Nivel de Transferencia de Registros, abstracción utilizada en el diseño de circuitos digitales que modela el flujo de señales digitales entre registros de hardware y las operaciones lógicas a realizar.

Resumen

Diseño e Implementación de un Acelerador de Hardware con FPGA para el Algoritmo de Redes Neuronales "Forward-Forward"

por

Yehohnathan Miranda Vigil

Universidad de Costa Rica

Facultad de Ingeniería

Escuela de Ingeniería Eléctrica

Julio del 2025

Este trabajo de graduación se centra en el desarrollo de un acelerador de hardware en una FPGA para el *Forward-Forward* (FF), con el objetivo de validar su eficiencia energética y capacidad computacional en tareas de clasificación de imágenes mediante una CNN. La problemática radica en la creciente demanda de recursos computacionales y el alto consumo energético asociado al entrenamiento y la inferencia de modelos de aprendizaje automático (*ML*), especialmente en sistemas convencionales como *CPU* y *GPU*, subrayando la necesidad de soluciones especializadas que equilibren eficiencia energética y rendimiento.

El objetivo general es desarrollar un acelerador de hardware que aproveche el paralelismo y la flexibilidad de las FPGAs, optimizando el FF para tareas de clasificación. La metodología incluye un enfoque cuantitativo basado en análisis experimental, utilizando herramientas como HLS de Vivado y SystemVerilog para el diseño, implementación y evaluación del sistema. Se espera como resultado un acelerador funcional de bajo consumo energético, sin comprometer el rendimiento, acompañado de un informe técnico, presentaciones de resultados y un diseño optimizado que sirva como base para futuras aplicaciones en IA y ML.

Palabras clave: *Forward-Forward, FPGA, CNN, ML, eficiencia energética*

Abstract

Design and Implementation of a Hardware Accelerator with FPGA for the "Forward-Forward" Neural Network Algorithm

by

Yehohnathan Miranda Vigil

University of Costa Rica
Faculty of Engineering
School of Electrical Engineering

July 2025

This graduation project focuses on developing a hardware accelerator on an FPGA for the *Forward-Forward* (FF) algorithm, aiming to validate its energy efficiency and computational capacity in image classification tasks using a *Convolutional Neural Network* (CNN). The problem arises from the increasing demand for computational resources and the high energy consumption associated with training and inference of *Machine Learning* (ML) models, especially in conventional systems such as *CPUs* and *GPUs*, emphasizing the need for specialized solutions that balance energy efficiency and performance.

The main goal is to develop a hardware accelerator leveraging the parallelism and flexibility of FPGAs, optimizing the FF for classification tasks. The methodology follows a quantitative approach based on experimental analysis, using tools such as Vivado HLS and SystemVerilog for design, implementation, and evaluation. The expected outcome is a fully functional accelerator with low power consumption, maintaining performance, accompanied by a technical report, result presentations, and an optimized design that can serve as a foundation for future applications in AI and ML.

Keywords: *Forward-Forward, FPGA, CNN, ML, energy efficiency*

1. Introducción

1.1. Contexto

El Aprendizaje Automático (ML) y la Inteligencia Artificial (AI) han revolucionado múltiples ámbitos en las últimas décadas, transformando no solo la ciencia y la tecnología, sino también el arte, la economía y la sociedad en general [1]. Lo que inició como planteamientos teóricos ha evolucionado hasta convertirse en herramientas esenciales para el análisis y la toma de decisiones a partir de grandes volúmenes de datos, sin la necesidad de contar con programación tradicional para realizar una tarea o tomar decisiones [2].

El ML, como rama fundamental de la IA, se basa en técnicas y algoritmos que permiten a las máquinas aprender de la experiencia [1]. Sus orígenes se remontan a los primeros trabajos pioneros de Alan Turing, quien en 1950 planteó la idea de que las máquinas podrían simular la inteligencia humana mediante el análisis de datos [4]. Posteriormente, Arthur Samuel desarrolló programas capaces de aprender y mejorar en juegos como las damas [5], sentando así las bases para el desarrollo de algoritmos de aprendizaje que se han perfeccionado con el paso del tiempo. La disponibilidad creciente de datos y el incremento en la capacidad computacional durante los últimos años han permitido que estos algoritmos se apliquen en una amplia variedad de campos, desde la medicina hasta la ingeniería.

La IA, por otro lado, es un campo interdisciplinario de conocimientos sobre informática, matemática, neurociencia y psicología; cuyo objetivo principal es la de diseñar sistemas capaces de realizar tareas complejas que históricamente han requerido de intervención humana, tales como, la toma de decisiones el reconocimiento de patrones, la resolución de problemas en entornos dinámicos, entre otras [1]. Una de las metodologías más consistentes para lograr este objetivo, son las Redes Neuronales Artificiales (ANN), que han cobrado especial relevancia por su capacidad de simular, en cierto punto, el funcionamiento de un cerebro humano [6].

[1] El ML mejora su desempeño ajustando parámetros internos a medida que acumula datos, experiencia. Sin embargo, la precisión llega a un límite llamado rendimiento asintótico, donde más datos ofrecen beneficios marginales debido a las limitaciones del modelo y la calidad de los datos [3].

Las ANN se inspiran en la estructura y funcionamiento de las neuronas biológicas, donde cada una recibe señales a través de sinapsis [2] y, a partir de la integración de estas señales, se produce una respuesta. La analogía "red neuronal", se remonta a los primero modelos teóricos presentados por McCulloch y Pitts en 1943, quieres propusieron una representación simplificada de actividad neuronal mediante circuitos lógicos [8]. Con el paso del tiempo, investigadores como Rumelhart, McClelland y *Hinton* profundizaron en el potencial de estos modelos, demostrando que la conexión ANN dividas por capas permite abordar problemas complejos de clasificación y reconocimiento [6]. Siendo actualmente las ANN la columna vertebral de muchas aplicaciones de ML, en áreas como el procesamiento de imágenes, el reconocimiento de voz y la traducción automática [9].

La conexión entre la biología y la ingeniería de las ANN ha sido muy importante, ya que aprovecha una sólida base teórica proveniente de la biología, especialmente del estudio sobre cómo el cerebro maneja y procesa la información [8]. Este conocimiento permite desarrollar sistemas avanzados que no solo se inspiran en la eficiencia y adaptabilidad del cerebro humano, sino que también buscan emular estas cualidades para resolver problemas complejos en el ámbito de la inteligencia artificial. Investigaciones, como las de Schidhuber [6], han demostrado que, al intentar emular la arquitectura y procesamiento del sistema nervioso, es posible crear modelos que no solo son altamente precisos, sino también capaces de aprendes de manera autónoma y a gran escala.

El perfeccionamiento de las ANN ha seguido el curso natural de los avances tecnológicos, dando lugar a aplicaciones que, años atrás, parecían imposibles. Ejemplos destacados de su implementación en la vida cotidiana incluyen Google Lens, con su avanzada capacidad para la detección precisa de objetos; Google Translator, que facilita la traducción en tiempo real mediante el reconocimiento de voz; los sistemas de recomendación predictivos, capaces de sugerir productos de manera casi mágica acorde a los intereses del usuario; y la detección de cáncer a través de imágenes médicas, logrando una notable precisión y eficiencia en el

[2] Conexión entre dos neuronas, caracterizada por un espacio diminuto que actúa como puente para facilitar la transmisión de información [7].

diagnóstico [1]. Sin embargo, estas aplicaciones dependen no solo de la ingesta masiva de datos, sino, de una creciente necesidad de capacidad de computo, y, de la calidad y eficiencia de los *algoritmos de entrenamiento* que ajustan los parámetros internos, los llamados pesos[3] [10].

Durante décadas, el algoritmo de retropropagación ha sido el método predominante para entrenar redes neuronales, revolucionando la forma en que las máquinas aprenden a partir de la experiencia [11]. Su funcionamiento se basa en propagar el error desde la capa de salida hacia las capas anteriores, lo que permite ajustar iterativamente los pesos de la red para minimizar la diferencia entre las predicciones y los resultados reales [10]. Este proceso iterativo ha permitido que las redes optimicen su desempeño progresivamente, logrando modelos cada vez más precisos, hasta un umbral. Sin embargo, pese a su éxito, la retropropagación presenta limitaciones importantes, como el problema del gradiente desvaneciente, que dificulta el entrenamiento de redes muy profundas, y la falta de coherencia con ciertos procesos biológicos de aprendizaje, lo que la hace poco plausible desde un punto de vista biológico [12].

Ante estas limitaciones, surge el algoritmo Forward-Forward (FF) como una alternativa prometedora. A diferencia de la retropropagación, el método FF entrena la red mediante dos pasadas hacia adelante, una con datos positivos y otra con datos negativos, lo que permite optimizar localmente cada capa sin recurrir al cálculo inverso de gradientes [13]. Esta metodología no solo evita el problema del gradiente desvaneciente, sino que resulta especialmente atractiva en entornos donde los recursos energéticos son limitados, como el Edge Computing [4], ya que mejora la eficiencia y reduce el consumo de energía en comparación con los métodos tradicionales de retropropagación [13].

Además, es fundamental no solo explorar alternativas en los algoritmos de entrenamiento, sino también adoptar soluciones innovadoras que satisfagan la demanda cada vez mayor de capacidad de cómputo en aplicaciones de redes neuronales. En este sentido, las Matrices de

[3] Son los valores numéricos que determinan la influencia de una entrada en la salida de una red neuronal, actuando como factores de ajuste.

[4] Se refiere al procesamiento de datos en dispositivos de bajo consumo ubicados en el "borde" de la red, cerca del usuario final, lo que permite respuestas en tiempo real sin depender de un centro de datos centralizado [14].

Compuertas Programables en Campo (FPGAs) emergen como una opción de hardware altamente eficiente, ofreciendo una alternativa sobresaliente frente a arquitecturas convencionales como las CPUs y GPUs [15], cuyos límites en eficiencia energética se hacen cada vez más evidentes ante el incremento exponencial de datos y la complejidad operativa. Una de las principales ventajas de las FPGAs radica en su capacidad para implementar arquitecturas personalizadas. Esto permite diseñar aceleradores que maximizan el paralelismo intrínseco de las redes neuronales [15], reduciendo significativamente el consumo energético. Gracias a su flexibilidad y reconfigurabilidad, estas plataformas se adaptan de forma precisa a las necesidades específicas de algoritmos innovadores como el FF, optimizando el flujo de datos y habilitando operaciones paralelas de manera altamente eficiente [16].

En un escenario en el que la sostenibilidad y el bajo consumo energético son prioridades esenciales, las FPGAs se posicionan como la solución ideal para potenciar el desempeño de las redes neuronales artificiales [15]. Estas plataformas permiten alcanzar un balance crucial entre un alto rendimiento y un consumo energético reducido, lo que resulta especialmente valioso para aplicaciones en dispositivos de bajo consumo y sistemas embebidos, sin sacrificar resultados de calidad y a la vez reduciendo costos de hardware.

La motivación detrás de este Trabajo Final de Graduación radica en la necesidad de desarrollar soluciones tecnológicas sostenibles que aborden los desafíos actuales de la inteligencia artificial. Con el diseño y la implementación de un acelerador de hardware con FPGA para el algoritmo FF, se busca no solo validar la eficiencia energética y el desempeño computacional en tareas de clasificación de imágenes, sino también promover el desarrollo de tecnologías accesibles y de bajo costo en Costa Rica, incentivando la investigación y la innovación en el ámbito de la IA.

1.2. Antecedentes

Toda investigación fundamentada en ideas preexistentes requiere una revisión exhaustiva de trabajos previos para construir una base sólida y robusta. Este proceso permite aprove-

char los avances y conocimientos acumulados, especialmente aquellos proporcionados por investigadores pioneros en el campo del procesamiento eficiente mediante plataformas reconfigurables, como las FPGAs. Este aspecto adquiere especial relevancia en el ámbito de la inteligencia artificial aplicada al hardware, donde las FPGAs han demostrado un notable potencial para equilibrar el rendimiento computacional y la eficiencia energética, superando en muchos casos a las CPUs y GPUs convencionales.

Dado que el enfoque de este trabajo se orienta al diseño e implementación de un acelerador de hardware basado en FPGA para aplicaciones de inteligencia artificial, se presenta una revisión de casos de éxito relevantes que brindan sustento teórico y técnico avalado por la comunidad científica. Uno de los ejemplos más destacados es la arquitectura **MALOC**, propuesta por Gong et al., la cual introduce un diseño completamente segmentado en pipeline, donde todas las capas de una CNN son mapeadas individualmente sobre el chip. Esta solución fue implementada en dos plataformas FPGA: alcanzó un rendimiento pico de 910.2 GOPS en la Virtex-7 690t y una eficiencia energética de 36.36 GOP/s/W en la Xilinx Zynq-7020. Estos resultados demuestran la viabilidad de soluciones on-chip que maximizan el paralelismo y minimizan el consumo energético, superando en eficiencia a enfoques tradicionales basados en arquitecturas ASIC estudiadas [17].

A partir de avances como este, diversos estudios han reafirmado la viabilidad de implementar aceleradores de redes neuronales en hardware. Un ejemplo destacado es la propuesta de Liang et al., quienes desarrollan FP-BNN, un acelerador para redes neuronales binarizadas que optimiza el uso de memoria interna mediante técnicas de cuantización y almacenamiento eficiente. Al sustituir las operaciones de multiplicación por operaciones lógicas de bajo costo, como XNOR y desplazamientos, FP-BNN logra reducir significativamente los accesos a memoria externa sin comprometer la precisión del modelo. Esto lo convierte en una alternativa altamente eficiente para dispositivos de bajo consumo energético, como las FPGAs, especialmente en aplicaciones embebidas [18]. Asimismo, Liu et al. presenta una arquitectura para CNNs en FPGAs que alcanza una eficiencia computacional del 97 %, y supera los 1.3 TOPS

en la plataforma Arria 10 GX1150, destacando su adaptabilidad a diferentes topologías [19].

Además, estudios realizados por Siddiqui y Naik han demostrado que, incluso en escenarios con recursos limitados, es factible diseñar aceleradores altamente eficientes utilizando la plataforma Virtex-7 [20]. Por otro lado, Yi et al. han desarrollado marcos de trabajo flexibles que facilitan una asignación equilibrada de los recursos de hardware entre las capas de una CNN. Este enfoque no solo optimiza el uso de procesadores de señales digitales, sino que también mejora significativamente el rendimiento general del sistema [21].

Un aspecto común entre estos estudios es su énfasis en el uso eficiente y especializado de los recursos reconfigurables intrínsecos a las FPGAs. Esto pone sobre la mesa que la implementación de aceleradores de hardware no solo es técnicamente viable, sino también esencial para desarrollar soluciones de alto rendimiento computacional con bajo consumo energético. En este contexto, la propuesta de utilizar una FPGA para implementar el algoritmo FF se encuentra respaldada por estas investigaciones, ya que este algoritmo se adapta perfectamente a la filosofía de trabajo óptima y eficiente que caracteriza a las FPGAs.

Adicionalmente, se cuenta con ecosistemas de desarrollo integrales como el propuesto por Leon-Vega et al., que puede llegar a facilitar significativamente la investigación y el diseño de aceleradores en FPGA. Dicho ecosistemas incluyen bibliotecas, flujos de trabajo y herramientas que reducen la complejidad técnica de democratizan el acceso a tecnologías de hardware avanzadas [22].

1.3. Planteamiento del Problema

El ML y la IA han transformado profundamente diversas áreas, permitiendo el desarrollo de aplicaciones avanzadas que han revolucionado desde la tecnología hasta la sociedad. Sin embargo, este progreso ha traído consigo un crecimiento exponencial en la complejidad y el tamaño de los modelos, lo que genera una demanda creciente de recursos computacionales. Esta mayor exigencia se traduce en un incremento significativo en el consumo energético y plantea desafíos económicos considerables. Por ejemplo, en el ámbito de la generación de

prompts[5], se ha estimado que, a inicios de 2024, un solo prompt puede producir aproximadamente 4,32 gramos de CO_2, consecuencia del alto consumo energético y la dependencia de fuentes de energía fósil [23].

En este contexto, el problema general que se busca atacar radica en optimizar la eficiencia energética sin comprometer el rendimiento computacional en el entrenamiento en modelos de ML, especialmente en tareas de clasificación de imágenes mediante CNN. Los métodos tradicionales de entrenamiento, como el algoritmo de retropropagación, han sido ampliamente utilizados en arquitecturas basadas en CPUs y GPUs; para todo tipo de modelos, incluido el famoso ChatGPT [24], sin embargo, estos enfoques presentan limitaciones significativas. Entre ellas, se destaca el elevado consumo energético, la inadecuación entre el hardware y el algoritmo, y problemas críticos como el gradiente desvaneciente, que no solo incrementan los costos operativos, sino que también generan un impacto ambiental considerable [12].

Ante estas barreras, se propone explorar alternativas que permitan reducir el consumo energético y mejorar la eficiencia en el entrenamiento de modelos, sin sacrificar la precisión. En este sentido, el algoritmo FF junto con el uso de una infraestructura de hardware, mediante un acelerador con una FPGA, capaz de soportar las exigencias de paralelismo y tener una mejor eficiencia energética. No obstante, la implementación de aceleradores basados en FPGAs para algoritmos como el FF requiere una serie de estrategias de diseño, optimización y validación que aún no se han explorado a fondo dentro del contexto local universitario.

Destacando que, en la Universidad de Costa Rica y por parte de la Escuela de Ingeniería Eléctrica, aún existe una brecha en el apoyo y la orientación para desarrollar iniciativas que integren aceleradores de hardware eficientes y sostenibles en el campo del ML. La presente investigación busca, además de validar el enfoque del algoritmo FF en una FPGA, fomentar el desarrollo de alternativas innovadoras que permitan alcanzar altos niveles de rendimiento computacional con un bajo consumo energético, alineándose con los valores de sostenibilidad y eficiencia que promueve la institución y el país.

[5]Mensajes o consultas enviados a modelos de lenguaje extenso (LLM), como ChatGPT, que dependen en gran medida del uso intensivo de GPUs.

1.4. Justificación

El desarrollo de un acelerador de hardware mediante FPGA para el algoritmo FF se justifica por la imperiosa necesidad de optimizar la eficiencia energética en aplicaciones de ML, sin sacrificar el rendimiento computacional. En un contexto global de creciente demanda de recursos y de preocupación por la sostenibilidad, es crucial explorar alternativas que logren un balance óptimo entre la potencia de procesamiento y el consumo energético. Desde una perspectiva técnica, la implementación del algoritmo FF, que evita limitaciones inherentes a métodos tradicionales como la retropropagación, permite explotar al máximo el paralelismo intrínseco de las FPGAs. Esta estrategia no solo optimiza el entrenamiento de redes neuronales en tareas de clasificación de imágenes mediante CNN, sino que podría minimizar los costos operativos y el impacto ambiental, aspectos cada vez más relevantes en el panorama actual.

Asimismo, el uso de FPGAs se destaca por su flexibilidad y reconfigurabilidad, lo que posibilita diseñar arquitecturas personalizadas adaptadas a las exigencias específicas de algoritmos innovadores como el FF. Este enfoque resulta especialmente valioso en dispositivos y sistemas embebidos de bajo consumo, donde la reducción del consumo energético es crítica al no existir energía ilimitada.

Desde el punto de vista del impacto y la contribución, este proyecto aportará una solución práctica y replicable para escenarios en los que se requiere procesamiento intensivo bajo restricciones energéticas. Además, al enfocarse en un contexto local, la Universidad de Costa Rica y la Escuela de Ingeniería Eléctrica, se sienta un precedente para futuras investigaciones y aplicaciones. Aunque en Costa Rica existen investigaciones relacionadas con el diseño de aceleradores de hardware, aún falta una guía y un apoyo institucional que integren estas soluciones de manera óptima, respetando los valores de sostenibilidad y eficiencia energética promovidos por la universidad.

Finalmente, la viabilidad de este trabajo se ve reforzada por metodologías integrales de desarrollo, como la propuesta de Leon-Vega et al., investigadores costarricenses, que simplifican el proceso de diseño y validación de aceleradores en FPGA. Así, el proyecto

no solo contribuirá con una solución de alto rendimiento y bajo consumo energético, sino que también servirá como un punto de apoyo y referencia para el desarrollo de tecnologías innovadoras y sostenibles en el ámbito de la inteligencia artificial.

1.5. Objetivos

Objetivo General

Diseñar e implementar un acelerador de hardware en una FPGA para el FF, con el objetivo de validar su eficiencia energética y capacidad computacional en tareas de clasificación de imágenes mediante una BNN y CNN , utilizando herramientas de optimización y paralelismo intrínsecas de una FPGA.

Objetivos Específicos

1. Diseñar la arquitectura del acelerador en la FPGA Nexys A7 utilizando Vivado HLS y el lenguaje SystemVerilog, maximizando el aprovechamiento de recursos y el paralelismo intrínseco del dispositivo.

2. Implementar y validar el algoritmo FF en una BNN, aprovechando la simplicidad de sus pesos para verificar la correcta ejecución del entrenamiento y la funcionalidad del acelerador.

3. Extender la implementación al diseño e implementación de una CNN, ajustando configuraciones de capas, hiperparámetros y funciones de activación, para enfrentar el mayor desafío que supone el manejo de pesos más complejos.

4. Evaluar el consumo energético (J) durante la inferencia de datos, utilizando mediciones obtenidas con una resistencia shunt y la función “Report Power” del software Vivado, y analizar su relación con el rendimiento computacional medido en operaciones por segundo.

1.6. Metodología

La metodología propuesta para llevar a cabo el presente proyecto se divide en cinco etapas, las cuales se explica general a continuación:

Etapa 1: Diseño de la arquitectura y preparación del entorno. En esta etapa se definirá la arquitectura del acelerador de hardware en la FPGA Nexys A7. Se utilizará Vivado HLS junto con SystemVerilog para desarrollar módulos personalizados que implementen el algoritmo FF. Además, se establecerán las métricas de rendimiento computacional y de consumo energético que servirán como referencia en las evaluaciones experimentales.

Etapa 2: Implementación y validación del algoritmo FF en una BNN. Se procederá a implementar el algoritmo FF en una BNN, aprovechando la simplicidad de sus pesos y operaciones. Este modelo inicial permitirá validar el correcto funcionamiento del entrenamiento en hardware, facilitando la detección de posibles errores y asegurando que la implementación base del FF sea robusta.

Etapa 3:Extensión de la implementación del algoritmo FF a una CNN. Una vez verificada la funcionalidad del FF en la BNN, se extenderá la implementación al caso de una red neuronal convolucional. Esta fase implica adaptar la arquitectura para gestionar pesos y operaciones de mayor complejidad propias de las CNN, integrando módulos específicos para realizar convoluciones, pooling y otras operaciones esenciales en la extracción y procesamiento de características en imágenes.

Etapa 4: Prueba de configuraciones distintas en la implementación de la CNN. Con la CNN implementada, se explorarán diversas configuraciones del modelo. Se variarán parámetros tales como el número de capas, filtros, hiperparámetros y funciones de activación para optimizar el diseño. Esta etapa experimental permitirá identificar la configuración que ofrezca el mejor equilibrio entre precisión, velocidad y eficiencia computacional.

Etapa 5: Evaluación del rendimiento y del consumo energético. Finalmente, se realizará una evaluación exhaustiva del acelerador. Se medirán métricas de rendimiento, utilizando conjuntos de datos estándar. Paralelamente, se evaluará el consumo energético durante la inferencia, comparando los resultados obtenidos en la implementación de la CNN con los datos recogidos inicialmente en la BNN, con el fin de analizar las mejoras en eficiencia y rendimiento alcanzadas por la optimización del diseño.

1.7. Organización del documento

2. Marco Teórico

2.1. Fundamentos de las Redes Neuronales Artificiales

2.1.1. Del perceptrón simple al perceptrón multicapa

En 1943, McCulloch y Pitts propusieron el primer modelo matemático inspirado en la neurona biológica[6], representando un paso fundamental para emular la capacidad cerebral de aprender a resolver problemas a partir de la experiencia. Este modelo empleaba lógica booleana[7] para realizar cálculos lógicos, estableciendo así las bases para el desarrollo de las ANN [25].

La unidad básica del sistema nervioso biológico es la neurona, cuya estructura principal está compuesta por tres elementos clave: el cuerpo neuronal, las dendritas y el axón (ver Figura 1). Al aumentar el número de neuronas, estas establecen conexiones entre sí mediante enlaces entre el axón de una neurona y las dendritas de otra, facilitando así la propagación de señales eléctricas. Sin embargo, la transmisión de estas señales no ocurre de forma indiscriminada; cada neurona debe cumplir ciertas condiciones para permitir o inhibir la propagación de señales eléctricas, regulando así cuáles *percepciones* son procesadas simultáneamente en el cerebro [26].

En 1958, Frank Rosenblatt consiguió replicar este comportamiento mediante un modelo matemático y computacional llamado **perceptrón**. En este modelo, los impulsos eléctricos de la neurona se interpretan como datos de entrenamiento, que son procesados y propagados a través del perceptrón para realizar tareas específicas de aprendizaje. Siendo la red neuronal artificial (ANN) más simple, el perceptrón puede aprender a clasificar patrones visuales ajustando sus pesos internos [25]. Su estructura, mostrada en la Figura 2, está compuesta por una capa de entradas $x_1, x_2, \ldots, x_n$, un único nodo de salida y un conjunto de conexiones internas denominadas pesos $w_1, w_2, \ldots, w_n$.

[6] Este logro se presentó en el artículo titulado *A Logical Calculus of the Ideas Immanent in Nervous Activity*.
[7] La lógica booleana se basa en dos valores posibles: verdadero (1) y falso (0).

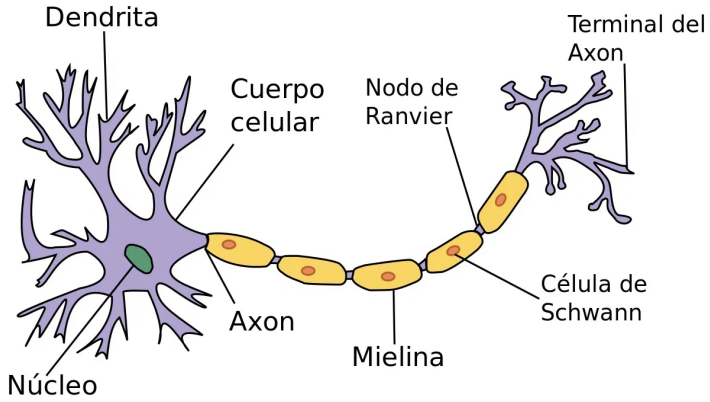

Figura 1. Estructura de una neurona [27].

La capa de entrada debe contener d nodos, correspondientes a las d características que describen cada instancia. En esta capa no se realizan operaciones de cálculo; únicamente se reciben los valores de entrada, los cuales posteriormente son multiplicados por sus respectivos pesos w_i, propagándose así la información hacia adelante [28]. Con esto, se calcula el producto que corresponde al nodo de salida

$$\overline{W} \cdot \overline{X} = \sum_{i=1}^{d} w_i x_i \tag{Ec.1}$$

Este producto tiene como resultado un número real, del cual se usa su **signo** para decidir sobre la clase que corresponde a la entrada. Por lo tanto, una predicción se puede definir como

$$\hat{y} = \text{signo}\left\{\overline{W} \cdot \overline{X}\right\} = \text{signo}\left\{\sum_{i=1}^{d} w_i x_i\right\} \tag{Ec.2}$$

Como se observa en las ecuaciones Ec.1 y Ec.2, los valores de salida que determinan las predicciones están directamente relacionados con los valores de los pesos internos. Ajustar adecuadamente estos pesos para maximizar la precisión de las predicciones es un aspecto fundamental del perceptrón y de la mayoría de las redes neuronales.

Al momento de realizar las predicciones el perceptrón, el tareas de clasificación, Rosenblatt tenia claro que habían errores, los cuales debían ser corregidos. Así, establece el error de

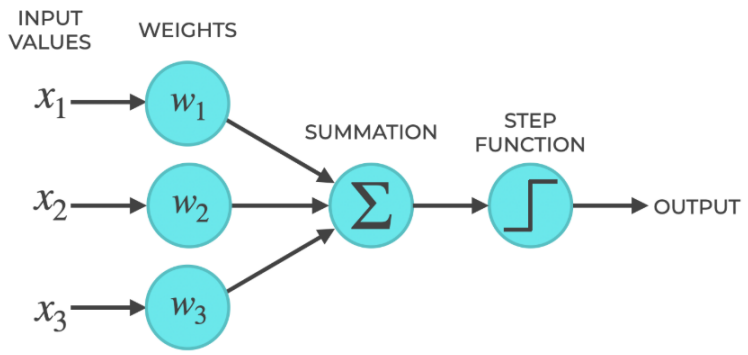

Figura 2. Estructura de un Perceptrón [29].

predicción como

Al realizar predicciones con el perceptrón en tareas de clasificación, Rosenblatt tenía claro que podían ocurrir errores, los cuales debían ser corregidos. Por lo tanto, definió el error de predicción como:

$$E\left(\overline{X}\right) = y - \hat{y} \tag{Ec.3}$$

donde y representa la predicción correcta, y los posibles resultados de esta ecuación son $-2, 0, 2$. Si el resultado era -2, significaba una predicción incorrecta para la clase A; si era 2, indicaba una predicción incorrecta para la clase B, y si era 0, la predicción era correcta [28]. Esta lógica estableció las bases del primer algoritmo de entrenamiento para una ANN, consistente en ajustar los pesos cuando la clasificación no era precisa.

A pesar de la innovación que supuso el modelo del perceptrón, este presentó limitaciones. Una de las más importantes fue su incapacidad para resolver problemas no linealmente separables[8], además de otras limitaciones destacadas por Minsky y Papert en 1969 [30].

Durante los años siguientes, las limitaciones del perceptrón condujeron a una marcada desaceleración en la investigación sobre ANN, un periodo conocido como el "invierno de la inteligencia artificial". No obstante, este panorama cambió significativamente a mediados de la

[8]Un problema no linealmente separable es aquel en el cual las diferentes clases de datos no pueden dividirse de forma precisa mediante una única línea o hiperplano.

década de 1980 con la introducción del algoritmo de *retropropagación*, que permitió entrenar redes neuronales con múltiples capas ocultas, conocidas como perceptrones multicapa (MLP) [25].

El MLP se compone de más de una capa en la que se realizan operaciones, y suele clasificarse como una red neuronal *hacia adelante*, ya que la información fluye únicamente en una dirección: desde la capa de entrada hasta la capa de salida. Cada neurona de una capa está conectada con todas las neuronas de la siguiente, permitiendo el procesamiento progresivo de los datos [31].

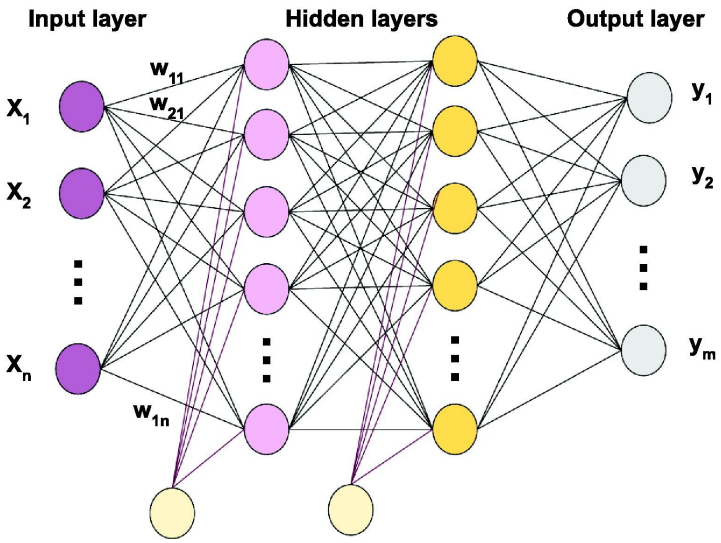

Figura 3. Estructura de un Perceptrón Multicapa [32].

Las capas que conforman una red MLP se describen a continuación:

- **Capa de entrada**: Compuesta por neuronas que reciben los datos iniciales del problema. No realiza procesamiento, solo transfiere la información hacia la siguiente capa.
- **Capas ocultas**: Formadas por neuronas que reciben señales de capas anteriores y transmiten sus salidas a capas posteriores. Aquí es donde se lleva a cabo el procesamiento complejo y la extracción de características.

- **Capa de salida**: Contiene las neuronas cuya activación representa la salida final de la red, correspondiente a la predicción del modelo.

Una vez definidos los hiperparámetros[9], el siguiente paso en el funcionamiento de una red MLP es el ajuste de los pesos internos, con el objetivo de obtener predicciones precisas. Para lograr esto, es indispensable contar con una cantidad suficiente de datos etiquetados, así como con un *algoritmo de entrenamiento* que permita optimizar los parámetros del modelo a partir de dichos datos [31].

2.1.2. Diversificación de las ANN

Después del desarrollo del modelo del perceptrón simple y el multicapa, se abrió el camino para la evolución y diversificaciones de las ANN. El MLP, como precursor fundamental de las ANN de múltiples capas, permitió profundizar en aspectos críticos como la definición de hiperparámetros, el ajuste de pesos y la aplicación de algoritmos de entrenamiento más sofisticados. Estas mejoras han facilitado el surgimiento de arquitecturas especializadas, cada una diseñada para abordar problemáticas particulares y optimizar el rendimiento en contextos específicos.

A partir del MLP se han derivado varios tipos de ANN, cada una adaptada a distintos tipos de datos y aplicaciones:

- **Redes Neuronales Espigadas (SNN):** Procesan la información mediante picos discretos, denominados *spikes*, imitando de manera más fiel la actividad neuronal biológica. Esta aproximación no solo mejora la similitud con el procesamiento cerebral, sino que también resulta en una mayor eficiencia energética, ya que el procesamiento basado en eventos permite realizar cálculos únicamente cuando se detecta una señal significativa. Las SNN son ideales para el manejo de datos espacio-temporales, como los generados

[9] Son valores de configuración establecidos antes del proceso de entrenamiento. Típicamente incluyen la cantidad de capas ocultas y el número de neuronas por capa.

por sensores dinámicos, y se utilizan en aplicaciones de *hardware neuromórfico*[10], donde se busca una simulación realista y energéticamente eficiente del procesamiento neuronal [33].

- **Redes Neuronales Recurrentes (RNN):** Diseñadas para procesar datos secuenciales y mantener información a lo largo del tiempo, lo que las hace ideales para aplicaciones como la traducción automática, el reconocimiento de voz y el modelado de series temporales. La característica recurrente permite que la red recuerdeïnformación de pasos anteriores, facilitando el procesamiento de secuencias en las que el orden de los datos es fundamental [31].

- **Redes Neuronales Generativas Adversarias (GAN):** Se componen de dos modelos que compiten entre sí: un generador y un discriminador. El generador intenta crear datos realistas, mientras que el discriminador evalúa la autenticidad de estos datos. Esta competencia permite generar contenido altamente realista, siendo ampliamente utilizadas en la síntesis de imágenes y la generación de datos sintéticos [34].

- **Redes Neuronales Binarizadas (BNN):** Tanto los pesos como las activaciones se representan de forma binaria. Esta simplificación reduce drásticamente la complejidad computacional y el consumo energético, facilitando su implementación en hardware [18].

- **Redes Neuronales Convolucionales (CNN):** Especializadas en el procesamiento de imágenes, estas redes utilizan capas de convolución para extraer características espaciales de los datos visuales y capas de pooling para reducir la dimensionalidad, facilitando así la clasificación y el reconocimiento de objetos [35].

[10] Se refiere a dispositivos diseñados para imitar la arquitectura y el funcionamiento del cerebro humano.

2.2. Redes Neuronales Binarias

Una **BNN** es un tipo especial de red de múltiples capas, también llamado Aprendizaje Profundo, en la que los pesos (y generalmente también las activaciones) se restringen a valores binarios, típicamente $-1, +1$ en vez de números reales continuos. Al forzar esta binarización, las BNN logran reducir drásticamente el almacenamiento y las operaciones requeridas, ya que las multiplicaciones matriz-vector pueden reemplazarse por simples operaciones bit a bit, como una XNOR (ver cuadro 1), seguidas de conteos de bits en 1 [18].

En hardware digital esto significa que los productos punto entre vectores binarios se efectúan con compuertas lógicas en vez de multiplicadores[11], logrando una aceleración significativa tanto en entrenamiento como en inferencia. De hecho, se ha demostrado que una BNN puede entrenarse hasta 7 veces más rápido en conjuntos como MNIST, manteniendo un desempeño cercano al de una red de precisión completa [36].

A	**B**	**A $\oplus$ B**
1	1	1
1	-1	-1
-1	1	-1
-1	-1	1

Cuadro 1. Tabla de verdad de la compuerta XNOR ($\oplus$) con valores 1 y -1, para una BNN.

Las BNN enfrentan un compromiso inherente entre eficiencia computacional y precisión. Al limitar los pesos y las activaciones a un solo bit, se sacrifica granularidad e información, lo que suele traducirse en una disminución de la exactitud en comparación con redes neuronales de mayor precisión. En conjuntos de datos grandes y complejos, como aquellos que involucran imágenes de alta resolución, las BNN tienden a obtener resultados menos precisos que arquitecturas más sofisticadas, como las CNN, requiriendo técnicas adicionales para cerrar esta brecha de rendimiento [37].

[11] Multiplicar $+1$ por $+1$ (o -1 por -1) produce $+1$, mientras $+1$ por -1 da -1. Codificando -1 como `bit 1` y -1 como `bit 0`, la multiplicación booleana equivale a una operación XNOR. La suma de muchos productos binarios se implementa entonces con un conteo de bits resultantes en 1.

Una de las principales mejoras propuestas para mitigar estas limitaciones fue XNOR-Net, una arquitectura que introdujo factores de escala en las capas binarizadas y reorganizó la estructura de la BNN. Este enfoque permitió reducir el error de cuantización, facilitando la ejecución de tareas más complejas con una pérdida mínima de precisión [37].

Todas estas optimizaciones responden a la necesidad de acercar el rendimiento de las BNN al de redes neuronales más complejas, manteniendo su principal ventaja: la eficiencia energética. Esto las hace especialmente adecuadas para su implementación en dispositivos de bajo consumo, donde el equilibrio entre desempeño y eficiencia resulta crítico.

Funcionamiento interno. Una BNN opera propagando los datos de entrada a través de capas binarizadas mediante un paso hacia adelante. En cada neurona, se calcula una suma ponderada de las entradas utilizando pesos w_i que solo pueden tomar los valores $+1$ o -1. Luego, se aplica una función de activación binaria (Ec.4), la cual devuelve $+1$si la suma total a es mayor o igual a cero, y -1 en caso contrario. De este modo, tanto las activaciones como las salidas de la red son binarias, lo que contribuye a una mayor eficiencia computacional [38].

$$b = \text{sign}(a) \tag{Ec.4}$$

Para entrenar a una BNN, comúnmente, se emplean estrategias especiales de retropropagación: en la práctica se utiliza un *estimador de derivadas por paso hacia delante (STE)*, que consiste en aproximar la derivada de la función escalón por una derivada no nula[12] [37]. En particular, las BNN suelen entrenarse mediante variaciones del gradiente descendiente como *Adam*, ajustando los pesos a partir de una versión latente en valores reales, que acumula los cambios pequeños antes de cuantizarlos [38].

En cada iteración de entrenamiento compara las salidas de la red con las etiquetas verdaderas del *conjunto de entrenamiento* usando una función de pérdida y actualiza los parámetros

[12]La función de activación signo no es diferenciable, lo que dificulta el uso de retropropagación. Para solucionarlo, se emplea el estimador STE, que aproxima su derivada con un valor no nulo, permitiendo el entrenamiento de la red.

minimizando dicho error. Es común que se usen técnicas como batch normalizado tras cada capa binaria, para estabilizar la distribución de activaciones antes de binarizarlas, mejorando la convergencia del entrenamiento [37].

Estructura e hiperparámetros. Una BNN puede adoptar arquitecturas similares a las de una ANN convencional. Esto incluye tanto capas completamente conectadas como estructuras más complejas basadas en convoluciones apiladas[13] [37].

Los principales hiperparámetros estructurales incluyen el número de capas, la cantidad de neuronas o filtros por capa y el tamaño de cada filtro (en caso de usar capas convolucionales), los cuales deben definirse en función de la tarea a resolver. De igual forma, otros hiperparámetros importantes son la tasa de aprendizaje del optimizador (por ejemplo, Adam), el tamaño del lote (*batch size*) y el número de épocas de entrenamiento[14] [37], los cuales deben ser seleccionados cuidadosamente para lograr un buen desempeño del modelo.

Otro hiperparámetro particular en BNN es el manejo de la escala de los pesos: dado que todos los pesos binarios tienen la misma magnitud, es común introducir coeficientes de escala aprendibles por capa (como en XNOR-Net). Aunque esto implica agregar algunos parámetros en punto flotante, puede mejorar notablemente la precisión del modelo sin comprometer significativamente su eficiencia computacional [37].

Aplicaciones. Las BNN se concentran en escenarios donde la eficiencia computacional es prioritaria. Un ejemplo, en procesamiento de imágenes se han utilizado BNN para tareas de detección, logrando ejecutar modelos de visión por computador en tiempo real sobre dispositivos móviles o embebidos con recursos limitados [39]. Un caso a resaltar, y que va de la mano con este proyecto de graduación, fue la clasificación utilizando el conjunto MNIST y de objetos CIFAR-10 con redes enteramente binarias, obteniendo exactitudes sobresalientes con solo 1 bit por peso [40].

[13] Se refiere a la disposición secuencial de múltiples capas convolucionales, donde cada capa procesa la salida de la anterior, permitiendo una extracción jerárquica de características.

[14] Una época se refiere a una pasada completa del modelo sobre todo el conjunto de datos de entrenamiento.

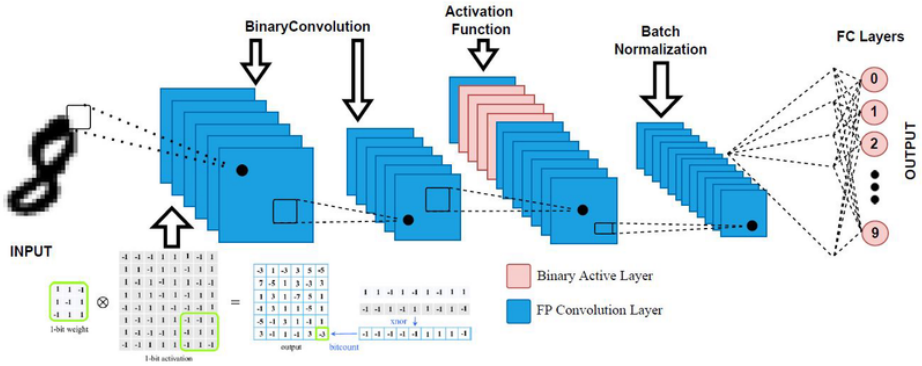

Figura 4. Estructura de una BNN [40].

2.3. Redes Neuronales Convolucionales

Las **CNN** son un tipo de arquitectura de Aprendizaje Profundo especialmente diseñada para procesar datos con estructura de rejilla, como imágenes, audio o señales 2D y 3D [35]. En una CNN típica, las neuronas están organizadas en *capas* que explotan la correlación espacial de los datos mediante la operación de *convolución* (de ahí su nombre).

Las operaciones de convolución consisten en desplazar un filtro (o kernel), ver figura 5, sobre la imagen de entrada (28x28 en la figura) y calcular productos punto locales para *extraer características* como bordes, texturas o formas. En el ejemplo ilustrado, el resultado de aplicar un filtro de 5x5 produce un mapa de 24x24 que resalta la presencia y ubicación de dichos patrones. A continuación, una capa de pooling (agrupamiento) reduce la dimensionalidad a 12x12, reteniendo la información más relevante y logrando invarianza a pequeñas traslaciones. Este proceso se repite en varias etapas, de modo que cada convolución y pooling sucesivo extrae y condensa características cada vez más abstractas. Finalmente, la salida se transforma en un vector de 192 dimensiones que alimenta una capa totalmente conectada (similar a una MLP), responsable de la clasificación del dígito con alta precisión. De esta forma, la red aprende a detectar patrones cada vez más complejos, pero mantiene la información esencial necesaria para la tarea de reconocimiento.

A grandes rasgos, una CNN se compone de tres tipos principales de capas: capas convo-

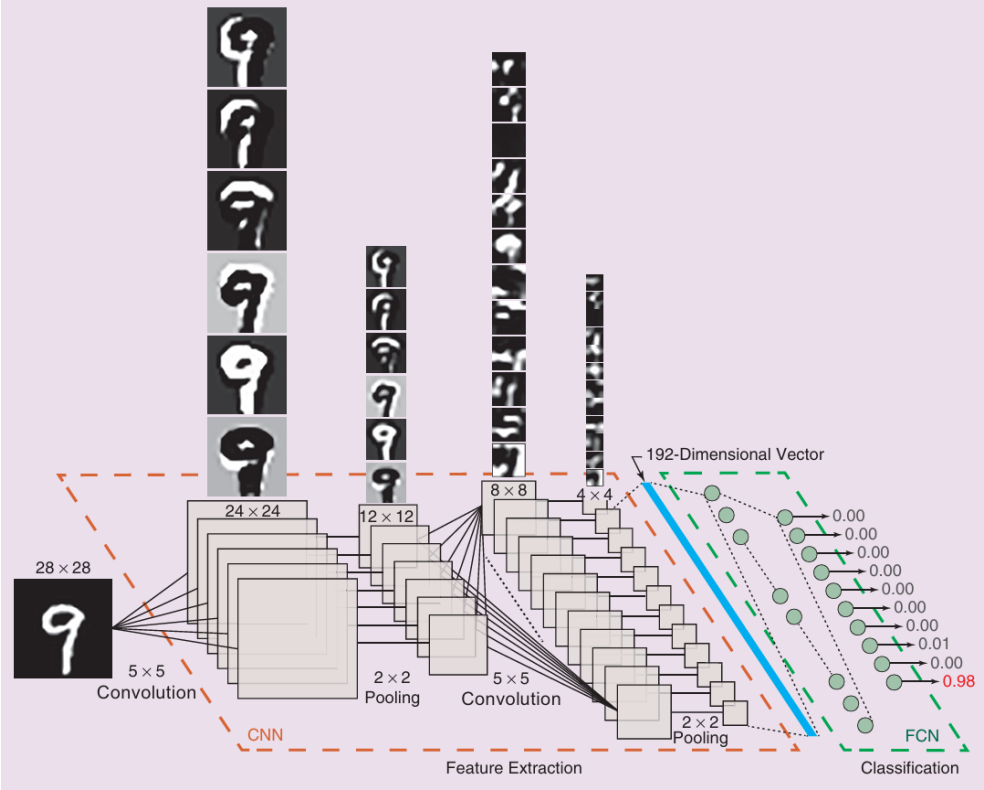

Figura 5. Una CNN entrenada para extraer características y clasificar números escritos a mano. El proceso incluye la aplicación de convoluciones sobre la imagen original del número nuevo, seguido de un pooling para reducir el tamaño de la convolución resultante [35].

lucionales, capas de pooling y capas completamente conectadas. Cada capa introduce una transformación incremental: las primeras capas convolucionales detectan características de bajo nivel (contornos) y las capas más profundas combinan estas características en patrones de más alto nivel (formas, objetos). Este enfoque jerárquico es uno de los rasgos distintivos de las CNN y les permite lograr resultados sobresalientes en visión por computadora.

Antes de la aparición de las CNN, el reconocimiento de imágenes dependía de extractores de características manuales seguidos de clasificadores; en contraste, las CNN aprenden automáticamente las características óptimas directamente de los datos, optimizando los filtros convolucionales mediante entrenamiento supervisado [41]. Este avance fue posibilitado por el algoritmo de **retropropagación**[15], que ajusta los pesos de la red minimizando una función

[15]A pesar de ser el algoritmo más utilizado, existe respaldo para la implementación del algoritmo Forward-

de pérdida general; esta actualiza los pesos de las convoluciones como los de las capas completamente conectadas usando gradientes calculados eficientemente a través de la *regla de la cadena*.

Un aspecto importante es que los pesos de cada filtro convencional están compartidos por todas las posiciones de la entrada, es decir, el mismo filtro (conjunto de pesos) se aplica en cada ubicación de la imagen, detectando la misma característica en diferentes regiones. Esta propiedad de *compartición de pesos* reduce drásticamente el número de parámetros en comparación con una red densa tradicional, ayudando a prevenir sobreajuste y aprovechando la naturaleza estacionaria de las estadísticas en imágenes (un borde puede aparecer en cualquier parte) [43].

Adicionalmente, las convoluciones son locales, cada neurona en la capa convolucionales solo recibe entradas de una pequeña vecindad (campo respectivo) de la capa anterior, lo que capta dependencia espacial de corto alcance y preserva la estructura topológica de los datos. Conforme se apilan más capas, el campo receptivo efectivo crece, permitiendo que la red responda a patrones más globales. De este modo, la arquitectura de una CNN está altamente estructurada para explotar la naturaleza de los datos de entrada, haciendo que sean muy eficientes.

Matemáticamente, la operación fundamental de una capa convencional es la **convolución discreta** entre el mapa de entrada y un filtro [44]. Si denotamos por X una entrada 2D (un digito de MNIST) y por K un filtro de tamaño $m \times n$, la convolución en una posición (i, j) produce:

$$Y(i,j);=;(X*K)(i,j);=;\sum_{u=0}^{m-1}\sum_{v=0}^{n-1}X(i+u,;j+v);K(u,v),. \qquad \text{(Ec.5)}$$

Cada neurona convolucional efectúa un cálculo que consiste en el producto elemento a elemento, seguido de una suma, aplicado sobre una región específica de la imagen. El resultado de este proceso genera un valor en el *mapa de características* de salida [44]. En las CNN

Forward en el entrenamiento de una CNN [42].

modernas, este procedimiento se realiza utilizando tensores tridimensionales, los cuales operan considerando parámetros adicionales como el alto, el ancho y los canales de la imagen, que permiten capturar información más compleja en un espacio de mayor dimensión.

En cada capa se aplica una **función de activación** no lineal a la salida de la convolución, antes o después del pooling. La activación más empleada, en CNN, es la *ReLU* [45], definida como:

$$\mathrm{ReLU}(z) = \mathrm{máx}(0, z) \tag{Ec.6}$$

La ReLU anula las respuestas negativas de una neurona y deja pasar las positivas, introduciendo no linealidad sin saturarse para valores grandes [43]. Haciendo que se sea muy útil para ANN de muchas capas, ya que busca evitar problemas como el gradiente desvaneciente. No obstante, la ReLU puede neuronas muertas que nunca si activan si el sesgo las lleva a región negativa permanente, para mitigar esto se usan variantes como Leaky ReLU.

Entrenamiento. Las CNN emplean un paradigma clásico de aprendizaje supervisado, que requiere un conjunto de entrenamiento compuesto por miles o incluso millones de imágenes etiquetadas. Inicialmente, los filtros de la red son configurados con valores aleatorios y, posteriormente, se ajustan iterativamente mediante un proceso de optimización dirigido a minimizar la función de pérdida. En cada iteración del algoritmo de entrenamiento, la red genera una salida basada en un minibatch de imágenes, la cual se compara con las etiquetas correspondientes para calcular el error. Este error se utiliza para actualizar los pesos de los filtros, refinando así el modelo para mejorar su desempeño en la tarea específica [35].

Para mitigar el riesgo de sobreentrenamiento (overfitting) causado por el gran número de parámetros que puede contener una CNN, se implementan diversas técnicas [35]. Entre ellas, se destaca el **dropout**[16] en las capas completamente conectadas. Además, se aplica la normalización por lotes después de las operaciones de convolución, estabilizando las

[16]Técnica que desactiva de forma aleatoria ciertas neuronas durante el entrenamiento, promoviendo una mayor generalización.

distribuciones de activaciones. También se generan variaciones de las imágenes del conjunto de entrenamiento mediante transformaciones como rotaciones, recortes, reflejos, entre otras, lo cual amplía la diversidad de los datos y mejora la robustez del modelo.

Implementación. Las operaciones de convolución en las CNN requieren una elevada cantidad de cálculos aritméticos, por lo que su entrenamiento generalmente se beneficia de aceleradores de hardware especializados [20]. El uso de GPUs desempeñó un papel crucial en el resurgimiento de las CNN, al permitir la ejecución de miles de multiplicaciones en paralelo con alta eficiencia. Sin embargo, en entornos de producción y optimización, es común recurrir a aceleradores específicos como los FPGA, que ofrecen flexibilidad y rendimiento optimizado para tareas específicas [46].

Actualmente, se han desarrollado metodologías avanzadas para utilizar FPGA como aceleradores. Por ejemplo, Liu et al. proponen un enfoque de aceleración "full-stack" [19], el cual aborda optimizaciones integrales que abarcan desde el diseño de la arquitectura de red hasta la síntesis en hardware, mejorando significativamente la eficiencia operativa.

2.4. Algoritmos de Entrenamiento: Retropropagación y Forward-Forward

2.4.1. Retropropagación

Es el algoritmo más utilizado para entrenar ANN. Este método de optimización basado en gradientes calcula la función de pérdida en la salida de la red y propaga el error hacia atrás a través de todas las capas, ajustando los pesos para minimizar dicho error. Matemáticamente, para una red con salida $\hat{y}$ y etiqueta real y, se define una función de pérdida $\mathscr{L}(y,\hat{y})$ y se actualizan los pesos $\mathbf{W}$ según la regla:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial \mathscr{L}}{\partial \mathbf{W}}, \tag{Ec.7}$$

donde η es la tasa de aprendizaje. La efectividad de este método en tareas como el reconocimiento de patrones y la clasificación lo ha convertido en el estándar en el entrenamiento de redes neuronales [47].

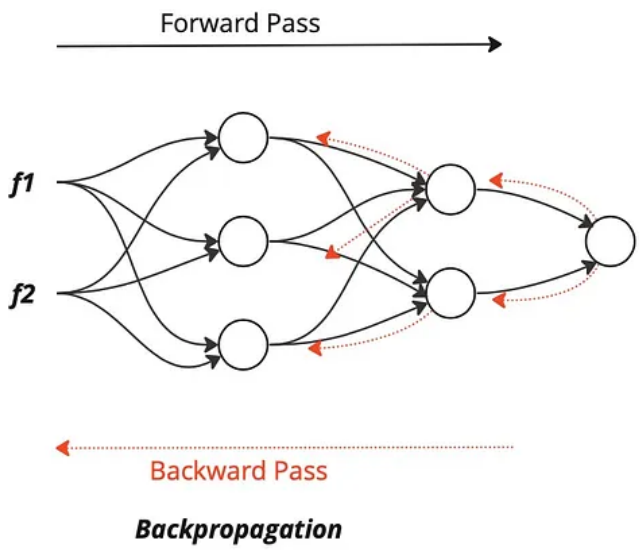

Figura 6. Algoritmo de entrenamiento: Retropropagación [48].

2.4.2. Forward-Forward: Una Alternativa al Entrenamiento Convencional

Propuesto por Geoffrey Hinton en 2022, presenta un enfoque novedoso para entrenar redes neuronales, diseñado como alternativa a la retropropagación. En lugar de utilizar un paso hacia atrás para propagar el error a través de la red, el método FF reemplaza la fase backward con dos pasadas únicamente hacia adelante: una con datos positivos reales y otra con datos negativos sintéticos [13]. Su objetivo es optimizar de manera local cada capa de la red maximizando la *bondad* de los datos positivos y minimizando la de los negativos, eliminando así la necesidad de calcular gradientes globales y facilitando la paralelización del entrenamiento.

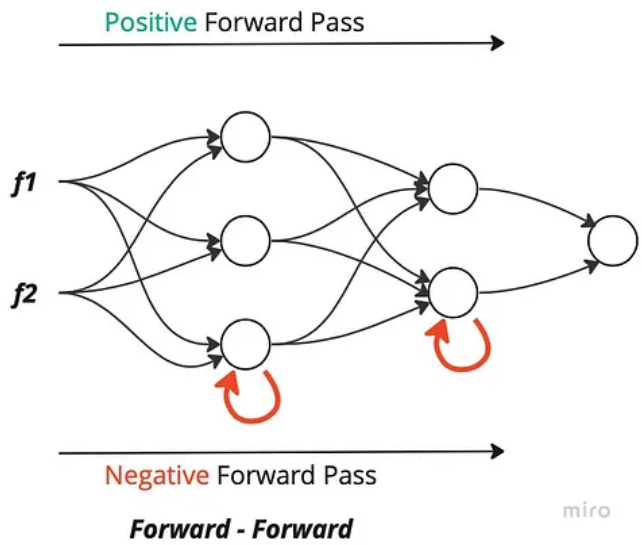

Figura 7. Algoritmo de entrenamiento: Forward-Forward [48].

En cada capa de la red, el algoritmo FF, evalúa una medida llamada *bondad* (G), definida generalmente como la suma de los cuadrados de las activaciones:

$$G = \|\mathbf{a}\|^2 = \sum_{i=1}^{n} a_i^2, \tag{Ec.8}$$

donde $\mathbf{a}$ representa el vector de activaciones de la capa. La idea es que, para una muestra positiva (datos reales), se desea que G sea mayor que un umbral ajustable θ, mientras que para una muestra negativa (datos sintéticos) se prefiere que G sea menor que θ.

Función de Pérdida Local. Para cada capa, se define una función de pérdida local que evalúa la discrepancia entre el valor de bondad G y el umbral θ. Una formulación común es utilizar funciones de pérdida basadas en la función sigmoide σ, de modo que:

$$L_{\text{pos}} = -\log \sigma(G - \theta) \quad \text{para datos positivos,} \tag{Ec.9}$$

$$L_{\text{neg}} = -\log\Big(1 - \sigma(G - \theta)\Big) \quad \text{para datos negativos,} \tag{Ec.10}$$

donde $\sigma(x) = \frac{1}{1+e^{-x}}$. De esta forma, se incentiva a la capa a producir activaciones de alta magnitud para entradas positivas y bajas para las negativas. La pérdida global en una capa, considerando N muestras (con $y_j = 1$ para datos positivos y $y_j = 0$ para negativos), se puede expresar como:

$$L = \frac{1}{N} \sum_{j=1}^{N} \left[y_j L_{\text{pos}}^{(j)} + (1 - y_j) L_{\text{neg}}^{(j)} \right]. \tag{Ec.11}$$

Esta función de pérdida se minimiza de forma local, permitiendo la actualización de los parámetros de cada capa sin recurrir a la propagación del error a lo largo de toda la red.

Pasadas Hacia Adelante y Aprendizaje Local. El proceso de entrenamiento con FF se ejecuta en dos pasadas únicamente hacia adelante:

1. **Datos Positivos:** Se introduce una muestra real $\mathbf{x}$ en la red. Para cada capa, se calcula la preactivación:

   $$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}, \tag{Ec.12}$$

   y se aplica una función de activación ϕ (por ejemplo, *Leaky ReLU*):

   $$\mathbf{a} = \phi(\mathbf{z}), \tag{Ec.13}$$

   donde:

   $$\phi(z_i) = \begin{cases} z_i, & \text{si } z_i > 0, \\ 0{,}01\, z_i, & \text{si } z_i \leq 0. \end{cases} \tag{Ec.14}$$

   Se calcula la bondad G y se busca maximizarla mediante la minimización de L_{pos}.

2. **Datos Negativos:** Se procesan datos sintéticos o alterados, siguiendo el mismo procedimiento. En este caso, se desea que la bondad G sea baja, minimizando L_{neg}.

Cada capa utiliza únicamente sus activaciones para actualizar sus parámetros mediante un optimizador (por ejemplo, Adam), lo que permite una **actualización local** y elimina

problemas comunes como el gradiente desvaneciente. Así, el algoritmo FF se caracteriza por una descentralización del aprendizaje, donde cada capa se entrena de manera autónoma y paralela.

Ventajas del Algoritmo Forward-Forward. El método FF ofrece varias ventajas significativas:

- **Paralelización:** Al operar de forma independiente, cada capa puede ser entrenada en paralelo, optimizando el uso de recursos computacionales.
- **Eliminación del Gradiente Desvaneciente:** La actualización local de cada capa evita la propagación de gradientes débiles a través de redes muy profundas.
- **Compatibilidad con Hardware Especializado:** Su estructura local y descentralizada facilita la implementación en plataformas como FPGAs o sistemas neuromórficos, maximizando la eficiencia energética.
- **Flexibilidad en la Medida de Bondad:** Aunque se ha utilizado la suma de los cuadrados de las activaciones, es posible explorar otras métricas que se adapten mejor a tareas específicas.

Aplicaciones y Avances. El algoritmo FF ha mostrado resultados prometedores en tareas supervisadas y no supervisadas. Por ejemplo, en la clasificación de imágenes se ha utilizado para entrenar redes alimentadas por conjuntos de datos como MNIST y CIFAR-10, alcanzando tasas de error comparables con las obtenidas mediante retropropagación. Además, su enfoque descentralizado lo hace especialmente atractivo para aplicaciones en dispositivos con recursos limitados, como microcontroladores y sistemas neuromórficos, donde la eficiencia computacional y energética es crucial [13].

2.5. Aceleradores de Hardware: Rol de las FPGAs

Definición y contexto. Un acelerador de hardware es un dispositivo diseñado para ejecutar tareas específicas con mayor rapidez y menor consumo energético en comparación con CPUs o GPUs. Estas ventajas se logran al implementar arquitecturas optimizadas que explotan el paralelismo inherente de las tareas computacionales intensivas, como las convoluciones y multiplicaciones de matrices en redes neuronales profundas [46].

Principales tipos de aceleradores de hardware. Entre los aceleradores más utilizados se encuentran:

- **CPU:** Aunque son versátiles y adecuadas para tareas generales, su rendimiento disminuye en aplicaciones especializadas debido a su arquitectura serial [49].
- **GPU:** Ofrecen un paralelismo masivo adecuado para aplicaciones de IA, pero su eficiencia energética es limitada y su flexibilidad arquitectónica es menor que la de las FPGAs, al tener que adaptarse a tareas generales en su conexto [49].
- **FPGA:** Destacan por su flexibilidad y capacidad para implementar arquitecturas específicas que maximizan el paralelismo y minimizan el consumo energético [50].

Ventajas de las FPGAs frente a otros aceleradores. En comparación con GPUs y CPUs, las FPGAs ofrecen:

- **Eficiencia energética:** Las FPGAs consumen hasta un orden de magnitud menos energía que las GPUs para tareas similares [18].
- **Flexibilidad arquitectónica:** Permiten personalizar los recursos de hardware para ajustarse a los requisitos específicos de un algoritmo o aplicación [17].
- **Reconfigurabilidad:** Su arquitectura puede ser reprogramada para diferentes aplicaciones, prolongando su utilidad en entornos con requisitos cambiantes [46].

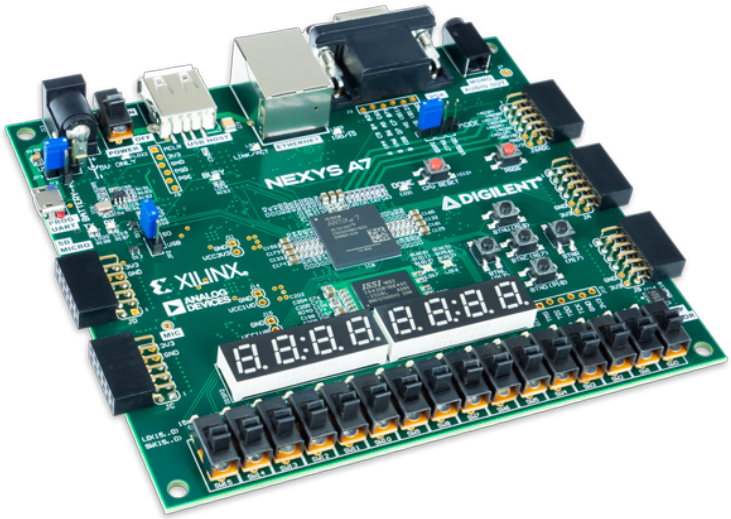

Figura 8. Artix-7 FPGA Nexys A7.

Rol de las FPGAs en inteligencia artificial y aprendizaje profundo. Las FPGAs se han adoptado ampliamente para acelerar tareas en inteligencia artificial debido a su capacidad para implementar estructuras optimizadas para CNNs [17] y redes neuronales binarizadas [18], entre otras tareas dentro del campo del ML. Estas aplicaciones se benefician del paralelismo masivo y la capacidad de reducir la latencia mediante arquitecturas pipelined y flujo de datos.

Características clave de las FPGAs. Las FPGAs están compuestas por bloques lógicos programables, interconexiones configurables y recursos específicos como DSPs y RAM embebida [50]. Entre sus características más relevantes se encuentran:

- **Paralelismo masivo:** Cada tarea puede ser descompuesta en suboperaciones que se ejecutan simultáneamente en múltiples bloques lógicos [46].
- **Optimización personalizada:** Herramientas como Vivado HLS permiten diseñar arquitecturas específicas que minimizan los recursos desperdiciados y maximizan el rendimiento [51].

- **Reducción de latencia:** Los pipelines y los flujos de datos optimizados disminuyen el tiempo de cómputo en operaciones repetitivas y complejas [52].

Guía para el diseño de un acelerador FPGA. El diseño de un acelerador en FPGA consta de varios pasos, los que se van a tomar en consideración para la elaboración de este Trabajo Final de Graduación, fundamentales basados en el ecosistema propuesto por Leon-Vega et al. [22]:

1. **Definición de los objetivos del acelerador:** Identificar las tareas específicas que el acelerador debe ejecutar. Esto incluye establecer métricas de desempeño como operaciones por segundo, consumo energético, y precisión.

2. **Selección de herramientas de diseño:** Utilizar herramientas como Vivado HLS para traducir algoritmos en arquitecturas de hardware optimizadas. Estas herramientas permiten convertir código en lenguajes de alto nivel, como C/C++, a implementaciones de bajo nivel optimizadas para FPGAs.

3. **Configuración del entorno de desarrollo:** Crear un entorno que incluya bibliotecas preconstruidas, motores de cuantización y plataformas de simulación para simplificar el diseño. Por ejemplo, el uso de APIs agnósticas y aceleradores reutilizables permite reducir el tiempo de implementación.

4. **Implementación de la arquitectura del acelerador:** Diseñar módulos específicos para operaciones clave como multiplicaciones de matrices, convoluciones y funciones de activación. Estos módulos deben configurarse para maximizar el paralelismo y minimizar el uso de recursos.

5. **Cuantización y optimización de datos:** Reducir la precisión de los datos utilizando motores de cuantización en tiempo de ejecución. Esto mejora la eficiencia energética y reduce el uso de memoria sin sacrificar significativamente la precisión.

6. **Simulación y pruebas:** Validar el diseño utilizando simulaciones que evalúen el consumo energético, el uso de recursos y el rendimiento. Herramientas como Vivado Power Report permiten estimar el consumo energético con precisión.

7. **Iteración y mejora:** Ajustar iterativamente el diseño basado en los resultados obtenidos en las simulaciones y pruebas experimentales. Esto incluye modificar la arquitectura, los parámetros de cuantización y los módulos de procesamiento según sea necesario.

8. **Despliegue en hardware:** Implementar el diseño final en una FPGA física, probando su desempeño en un entorno real. Durante esta etapa, es esencial monitorear el consumo energético y las métricas de desempeño para validar el cumplimiento de los objetivos iniciales.

3. Desarrollo del Proyecto

3.1. Evaluación Integral del Acelerador de Hardware

3.1.1. Calidad en Clasificación de Redes Neuronales

- **Exactitud (Accuracy):** Representa la fracción de predicciones totales, positivas y negativas, que el modelo acierta. Se calcula como la suma de verdaderos positivos y verdaderos negativos ($TP+TN$) dividida entre todas las muestras evaluadas ($TP+TN+FP+FN$):

$$\text{accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \tag{Ec.15}$$

  Una exactitud alta indica que el modelo clasifica correctamente la mayor parte de las entradas.

- **Precisión (Precision):** Indica qué tan confiables son las salidas positivas del clasificador: de todo lo que la red etiquetó como "positivo", ¿cuánto era realmente positivo? Se calcula dividiendo los verdaderos positivos (TP) entre el total de predicciones positivas ($TP+FP$):

$$\text{precision} = \frac{TP}{TP+FP} \tag{Ec.16}$$

- **Recuperación (Recall):** Mide la capacidad del modelo para "atrapar" todos los casos positivos: de todos los positivos reales, ¿qué fracción detectó? Se define como relación entre TP y la suma $TP+FN$:

$$\text{recall} = \frac{TP}{TP+FN} \tag{Ec.17}$$

- **F1-Score:** Es la media armónica entre precisión y recuperación:

$$\text{F1-Score} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \tag{Ec.18}$$

Un F1-Score cercano a 1 significa que tanto la precisión como la recuperación son altas.

3.1.2. Rendimiento: Throughput, Latencia y Recursos

- **Throughput (Operaciones por segundo):** Cuantifica la capacidad del sistema para procesar entradas por unidad de tiempo. En el contexto específico de los modelos BNN y CNN que utilizan imágenes, esta métrica suele expresarse en términos de imágenes por segundo durante las fases de entrenamiento e inferencia, o en GOPS. Matemáticamente, se define mediante la relación entre el número de imágenes (o de operaciones) completadas satisfactoriamente ($N_{\text{procesado}}$) y el intervalo de tiempo requerido (t):

$$\text{Throughput} = \frac{N_{\text{procesado}}}{t} \quad [\text{img/s o GOPS}] \tag{Ec.19}$$

  Esta métrica permite evaluar de manera objetiva y cuantitativa el desempeño computacional del acelerador de hardware implementado en FPGA.

- **Latencia:** Corresponde al intervalo temporal transcurrido desde el instante en que una muestra ingresa al acelerador de hardware hasta el momento en que éste produce una salida válida. En el contexto de modelos como BNN y CNN, la latencia puede definirse como el tiempo necesario para procesar completamente una imagen desde la capa de entrada hasta la obtención del resultado final, como una actualización de los pesos mediante el algoritmo FF. En arquitecturas de hardware altamente pipelinizadas la latencia se calcula mediante la relación directa entre el número total de etapas del pipeline (N_{etapas}) y el periodo del reloj utilizado (T_{clk}):

$$\text{Latencia} = N_{\text{etapas}} \times T_{\text{clk}} \tag{Ec.20}$$

  La latencia es fundamental para evaluar el desempeño temporal y la eficiencia operativa

del acelerador en FPGA, permitiendo identificar el tiempo mínimo de respuesta del sistema.

- **Utilización de recursos de la FPGA**: Porcentaje ocupado de cada tipo de bloque interno:

$$\text{Uso}_X = \frac{\text{Bloques } X_{\text{utilizados}}}{\text{Bloques } X_{\text{totales}}} \times 100 \qquad \text{(Ec.21)}$$

con $X \in \{\text{LUT}, \text{FFLOP}, \text{BRAM}, \text{DSP}\}$. Permitiendo ver la eficiencia del mapeo del hardware y entender limitaciones de escalabilidad. Además, de poder observar la diferencia de recursos que se destinan para una BNN y CNN.

3.1.3. Eficiencia Energética en la FPGA

- **Potencia estática (P_{stat}):** Es la potencia que el FPGA consume cuando no está realizando actividad de conmutación. En otras palabras, se trata del consumo debido a corrientes de fuga cuando el dispositivo esta en reposo, en la Nexys A7 depende de factores de temperatura, tensión de alimentación y características de fabricación del chip. Su importancia radica en fijar el *piso* de consumo que comparte cualquier diseño (BNN o CNN) la FPGA.

- **Potencia dinámica (P_{dyn}):** Corresponde a la potencia consumida por la FPGA como resultado de la conmutación de señales eléctricas (cambios de estado en los transistores) y la carga y descarga de capacitancias internas durante el funcionamiento activo del diseño implementado.

$$P_{\text{dyn}} = \alpha C V_{DD}^2 f \qquad \text{(Ec.22)}$$

donde α representa la tasa media de conmutación de señales internas, C es la capacitancia involucrada, V_{DD} indica la tensión de alimentación del dispositivo, y f corresponde a la frecuencia del reloj del sistema.

Para determinar esta métrica se emplean técnicas experimentales, como la diferencia entre la potencia total medida y la potencia estática, o la herramienta *Vivado Power Analysis*, disponible en la Nexys A7.

- **Energía por operación ($E_{\mathbf{op}}$):** Es la energía consumida para realizar una operación fundamental (por ejemplo, actualización de pesos).

$$E_{\text{op}} = \frac{P_{\text{tot}}}{\text{ops}\,/\,\text{s}} = P_{\text{tot}} \times t_{\text{op}} \tag{Ec.23}$$

(equivalentemente $E_{\text{op}} = P_{\text{tot}} \times t_{\text{op}}$). Dada la simplifica de BNN frente a CNN, se espera una menor energía de operación de esta. Además, será útil para observar el progreso en la optimización del entrenamiento.

- **Medición práctica con resistencia *shunt*: DETALLAR MÁS ADELANTE LA CREACIÓN DEL CIRCUITO CON LA RESISTENCIA DE SHUNT** Consiste en implementar un circuito de sensado mediante una resistencia de derivación (*shunt*), colocada en serie con la línea de alimentación de la Nexys A7, con el objetivo de cuantificar experimentalmente el consumo energético real del diseño implementado. Este método permite obtener mediciones directas y precisas de la corriente consumida por el acelerador, lo cual es fundamental para validar las estimaciones teóricas obtenidas con Vivado Power Analysis.

3.2. Diseño de la Arquitectura con HLS

4. Conclusiones y recomendaciones

Bibliografía

[1] S. M. Mian, M. S. Khan, M. Shawez y A. Kaur, "Artificial Intelligence (AI), Machine Learning (ML) & Deep Learning (DL): A Comprehensive Overview on Techniques, Applications and Research Directions," en *2024 2nd International Conference on Sustainable Computing and Smart Systems (ICSCSS)*, 2024, págs. 1404-1409. DOI: 10.1109/ICSCSS60660.2024.10625198.

[2] B. Quinto, "Introduction to Machine Learning," *Next-Generation Machine Learning with Spark*, 2020. DOI: 10.1007/978-1-4842-5669-5_1.

[3] A. Ramaswamy y E. Hüllermeier, "Deep Q-Learning: Theoretical Insights From an Asymptotic Analysis," *IEEE Transactions on Artificial Intelligence*, vol. 3, págs. 139-151, 2022. DOI: 10.1109/tai.2021.3111142.

[4] E. Brynjolfsson, "The Turing Trap: The Promise & Peril of Human-Like Artificial Intelligence," *Daedalus*, vol. 151, págs. 272-287, 2022. DOI: 10.1162/daed_a_01915.

[5] J. Schaeffer y H. van den Herik, "Games, Computers, and Artificial Intelligence," *Artificial Intelligence*, vol. 134, págs. 1-7, 2002. DOI: 10.1016/S0004-3702(01)00165-5.

[6] J. Schmidhuber, "Annotated History of Modern AI and Deep Learning," *ArXiv*, vol. abs/2212.11279, 2022. DOI: 10.48550/arXiv.2212.11279.

[7] M. Tsodyks, K. Pawelzik y H. Markram, "Neural Networks with Dynamic Synapses," *Neural Computation*, vol. 10, págs. 821-835, 1998. DOI: 10.1162/089976698300017502.

[8] I. Alzoubi, G. Bao, Y. Zheng, X. Wang y M. Graeber, "Artificial intelligence techniques for neuropathological diagnostics and research," *Neuropathology*, vol. 43, 2022. DOI: 10.1111/neup.12880.

[9] I. Devi, G. R. Karpagam y B. V. Kumar, "A survey of machine learning techniques," vol. 3, pág. 203, 2017. DOI: 10.1504/IJCSYSE.2017.10010099.

[10] S. K. y S. S, "REVIEW ON CLASSIFICATION BASED ON ARTIFICIAL NEURAL NETWORKS," vol. 2, págs. 11-18, 2014. DOI: 10.5121/IJASA.2014.2402.

[11] R. G. Gironés, J. Cerdá, F. Ballester-Merelo y A. Salcedo, "Artificial neural network implementation on a single FPGA of a pipelined on-line backpropagation," *Proceedings 13th International Symposium on System Synthesis*, págs. 225-230, 2000. DOI: 10.1145/501790.501837.

[12] H. Elsayed, B. U. Pedroni, S. Sheik y G. Cauwenberghs, "Hardware-Efficient On-line Learning through Pipelined Truncated-Error Backpropagation in Binary-State Networks," *Frontiers in Neuroscience*, vol. 11, 2017. DOI: 10.3389/fnins.2017.00496.

[13] G. E. Hinton, "The Forward-Forward Algorithm: Some Preliminary Investigations," *ArXiv*, vol. abs/2212.13345, 2022. DOI: 10.48550/arXiv.2212.13345.

[14] S. Deng, H. Zhao, J. Yin, S. Dustdar y A. Y. Zomaya, "Edge Intelligence: The Confluence of Edge Computing and Artificial Intelligence," *IEEE Internet of Things Journal*, vol. 7, págs. 7457-7469, 2019. DOI: 10.1109/JIOT.2020.2984887.

[15] R. G. Biradar, A. Chatterjee, P. Mishra y K. George, "FPGA implementation of a multilayer Artificial Neural Network using System-on-Chip design methodology," *2015 International Conference on Cognitive Computing and Information Processing (CCIP)*, págs. 1-6, 2015. DOI: 10.1109/CCIP.2015.7100683.

[16] A. Momeni, B. Rahmani, M. Malléjac, P. del Hougne y R. Fleury, "Backpropagation-free training of deep physical neural networks," *Science*, vol. 382, págs. 1297-1303, 2023. DOI: 10.1126/science.adi8474.

[17] L. Gong, C. Wang, X. Li, H.-p. Chen y X. Zhou, "MALOC: A Fully Pipelined FPGA Accelerator for Convolutional Neural Networks With All Layers Mapped on Chip," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, págs. 2601-2612, 2018. DOI: 10.1109/TCAD.2018.2857078.

[18] S. Liang, S. Yin, L. Liu, W. Luk y S. Wei, "FP-BNN: Binarized neural network on FPGA," *Neurocomputing*, vol. 275, págs. 1072-1086, 2018. DOI: 10.1016/j.neucom.2017.09.046.

[19] S. Liu, H. Fan, M. Ferianc, X. Niu, H. Shi y W. Luk, "Toward Full-Stack Acceleration of Deep tional Neural Networks on FPGAs," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, págs. 3974-3987, 2021. DOI: 10.1109/TNNLS.2021.3055240.

[20] A. F. Siddiqui y P. B. R. Naik, "Implementation of FPGA-based Accelerator for Convolutional Neural Networks," *April-May 2024*, 2024. DOI: 10.55529/ijrise.43.10.16.

[21] Q. Yi, H. Sun y M. Fujita, "FPGA Based Accelerator for Neural Networks Computation with Flexible Pipelining," *ArXiv*, vol. abs/2112.15443, 2021.

[22] L. G. Leon-Vega, E. Obregon-Fonseca y J. Castro-Godinez, "A User-Friendly Ecosystem for AI FPGA-Based Accelerators," en *2024 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*, jul. de 2024, págs. 1-6. DOI: 10.1109/COINS61597.2024.10622147.

[23] Smartly.ai. "The Carbon Footprint of ChatGPT: How Much CO2 Does a Query Generate?" Accessed: 2025-03-28. (2023), dirección: https://smartly.ai/blog/the-carbon-footprint-of-chatgpt-how-much-co2-does-a-query-generate?utm_source=chatgpt.com.

[24] P. Bhave, R. Chopade, S. Pawar y A. Stanam, "OpenAI's GPT-3/ChatGPT: Suggests & Correct," *2024 International Conference on Electrical, Computer and Energy Technologies (ICECET)*, págs. 1-4, 2024. DOI: 10.1109/ICECET61485.2024.10698683.

[25] N. Yadav, A. Yadav y M. Kumar, "History of Neural Networks," págs. 13-15, 2015. DOI: 10.1007/978-94-017-9816-7_2.

[26] S. Zarazua, "Artificial Neural Networks | Encyclopedia," 2020.

[27] Enciclopedia Significados, *Neurona: Qué es, Partes, Función y Tipos (con Dibujo)*, Accedido: 28 de marzo de 2025, 2024. dirección: https://www.significados.com/neurona/.

[28] S. Dreyfus, "Artificial neural networks, back propagation, and the Kelley-Bryson gradient procedure," *Journal of Guidance, Control, and Dynamics*, vol. 13, n.º 5, págs. 926-928, 1990. DOI: 10.2514/3.25422.

[29] J. Ebner, *Perceptrons, Explained*, 2023. dirección: https://www.sharpsightlabs.com/blog/perceptrons-explained/.

[30] M. Minsky y S. Papert, *Perceptrons: An Introduction to Computational Geometry*, 1st. Cambridge, MA: MIT Press, 1969, ISBN: 9780262130455.

[31] N. Kriegeskorte y T. Golan, "Neural network models and deep learning," *Current Biology*, vol. 29, R231-R236, 2019. DOI: 10.1016/j.cub.2019.02.034.

[32] ScienceDirect, *Multilayer Perceptron*, Accedido: 28 de marzo de 2025, s.f. dirección: https://www.sciencedirect.com/topics/computer-science/multilayer-perceptron.

[33] A. Tavanaei, M. Ghodrati, S. R. Kheradpisheh, T. Masquelier y A. Maida, "Deep Learning in Spiking Neural Networks," *Neural networks : the official journal of the International Neural Network Society*, vol. 111, págs. 47-63, 2018. DOI: 10.1016/j.neunet.2018.12.002.

[34] A. Creswell, T. White, V. Dumoulin, K. Arulkumaran, B. Sengupta y A. Bharath, "Generative Adversarial Networks: An Overview," *IEEE Signal Processing Magazine*, vol. 35, págs. 53-65, 2017. DOI: 10.1109/MSP.2017.2765202.

[35] R. C. Gonzalez, "Deep Convolutional Neural Networks [Lecture Notes]," *IEEE Signal Processing Magazine*, vol. 35, págs. 79-87, 2018. DOI: 10.1109/MSP.2018.2842646.

[36] M. Biswal, T. S. Delwar, A. Siddique, P. Behera, Y. Choi y J. Ryu, "Pattern Classification Using Quantized Neural Networks for FPGA-Based Low-Power IoT Devices," *Sensors*, vol. 22, n.º 22, pág. 8694, 2022. DOI: 10.3390/s22228694.

[37] Y. Su, K. P. Seng, L. M. Ang y J. Smith, "Binary Neural Networks in FPGAs: Architectures, Tool Flows and Hardware Comparisons," *Sensors*, vol. 23, n.º 22, 2023, ISSN: 1424-8220. DOI: 10.3390/s23229254. dirección: https://www.mdpi.com/1424-8220/23/22/9254.

[38] T. Agrawal, "Binarized Neural Network (BNN) and Its Implementation in Machine Learning," *Neptune.ai Blog*, 2023, Accedido: 28 de marzo de 2025. dirección: https://neptune.ai/blog/binarized-neural-network-bnn-and-its-implementation-in-ml.

[39] G. Zhu, H. Fei, J. Hong, Y. Luo y J. Long, "An Information-Reserved and Deviation-Controllable Binary Neural Network for Object Detection," *Mathematics*, vol. 11, n.º 1, pág. 62, 2023. DOI: 10.3390/math11010062. dirección: https://www.mdpi.com/2227-7390/11/1/62.

[40] M. Biswal, T. Delwar, A. Siddique, P. Behera, Y. Choi y J.-Y. Ryu, "Pattern Classification Using Quantized Neural Networks for FPGA-Based Low-Power IoT Devices," *Sensors*, vol. 22, pág. 8694, nov. de 2022. DOI: 10.3390/s22228694.

[41] IBM, *¿Qué son las redes neuronales convolucionales?* Accedido: 3 de enero de 2025, 2024. dirección: https://www.ibm.com/es-es/topics/convolutional-neural-networks.

[42] R. Scodellaro, A. Kulkarni, F. Alves y M. Schröter, "Training Convolutional Neural Networks with the -Forward algorithm," *ArXiv*, vol. abs/2312.14924, 2023. DOI: 10.48550/arXiv.2312.14924.

[43] R. Yamashita, M. Nishio, R. K. G. Do y K. Togashi, “Convolutional neural networks: an overview and application in radiology,” *Insights Imaging*, vol. 9, n.º 4, págs. 611-629, 2018. DOI: 10.1007/s13244-018-0639-9. dirección: https://doi.org/10.1007/s13244-018-0639-9.

[44] K. Ghiasi-Shirazi, “Generalizing the Convolution Operator in Convolutional Neural Networks,” *Neural Processing Letters*, vol. 50, n.º 3, págs. 2627-2646, 2019. DOI: 10.1007/s11063-019-10043-7.

[45] S. Rallabandi, *Activation functions: ReLU vs. Leaky ReLU*, Accedido: 28 de noviembre de 2024, 2023. dirección: https://medium.com/@sreeku.ralla/activation-functions-relu-vs-leaky-relu-b8272dc0b1be.

[46] S. Mittal, “A survey of FPGA-based accelerators for convolutional neural networks,” *Neural Computing and Applications*, vol. 32, págs. 1109-1139, 2018. DOI: 10.1007/s00521-018-3761-1.

[47] P. Werbos, “Backpropagation Through Time: What It Does and How to Do It,” *Proc. IEEE*, vol. 78, págs. 1550-1560, 1990. DOI: 10.1109/5.58337.

[48] S. Technology, *The Forward-Forward Algorithm*, Accedido: 28 de noviembre de 2024, 2023. dirección: https://medium.com/sahibinden-technology/the-forward-forward-algorithm-227479c76fa8.

[49] U. Rueckert, “Digital Neural Network Accelerators,” págs. 181-202, 2020. DOI: 10.1007/978-3-030-18338-7_12.

[50] A. Boutros y V. Betz, “FPGA Architecture: Principles and Progression,” *IEEE Circuits and Systems Magazine*, vol. 21, págs. 4-29, 2021. DOI: 10.1109/MCAS.2021.3071607.

[51] T. Cao, “Implementation and Research of Vivado HLS’s Function on FPGA,” *2022 3rd International Conference on Computer Science and Management Technology (ICCSMT)*, págs. 240-243, 2022. DOI: 10.1109/iccsmt58129.2022.00057.

[52] C. Zhu, K. Huang, S. Yang, Z. Zhu, H. Zhang y H. Shen, “An Efficient Hardware Accelerator for Structured Sparse Convolutional Neural Networks on FPGAs,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, págs. 1953-1965, 2020. DOI: 10.1109/TVLSI.2020.3002779.